

쓰기 작업시 WAL 병목과 디스크 I/O 경합에 따 른 RocksDB 최적화

RocksDB Study 희로애락

윤치호 박규원 강민수 이우진

2026. 02. 04

Contents

1. Overview

2. Background

3. Hypothesis

4. Experiment

5. Result & discussion

Overview - 병목 현상 분석

핵심 목표

- 고성능 SSD 환경에서 Disk I/O 경합으로 발생하는 **병목 현상**을 분석한다.
- 3가지 핵심 튜닝 옵션의 효과를 측정하여 **처리량(OPS)**과 **지연 시간(P99.9)**의 **최적 조합**을 도출한다.

옵션	효과
로그 전용 디스크 분리	디스크 I/O 경합 제거 → p99 개선 가능
wal_bytes_per_sync 증가	fsync 빈도 감소 → fsync 오버헤드 완화
compaction rate limiter	compaction 쓰기를 제어 → WAL 대기 완화

Overview - 병목 현상 분석

로그 전용 디스크 분리

WAL append는 작은 랜덤/append I/O가 많아 대형 순차 쓰기 (compaction)와 큐 경쟁 시 p99가 급증한다.

별도 디스크에 WAL을 두면 큐 경쟁을 물리적으로 분리해 즉시 효과를 본다

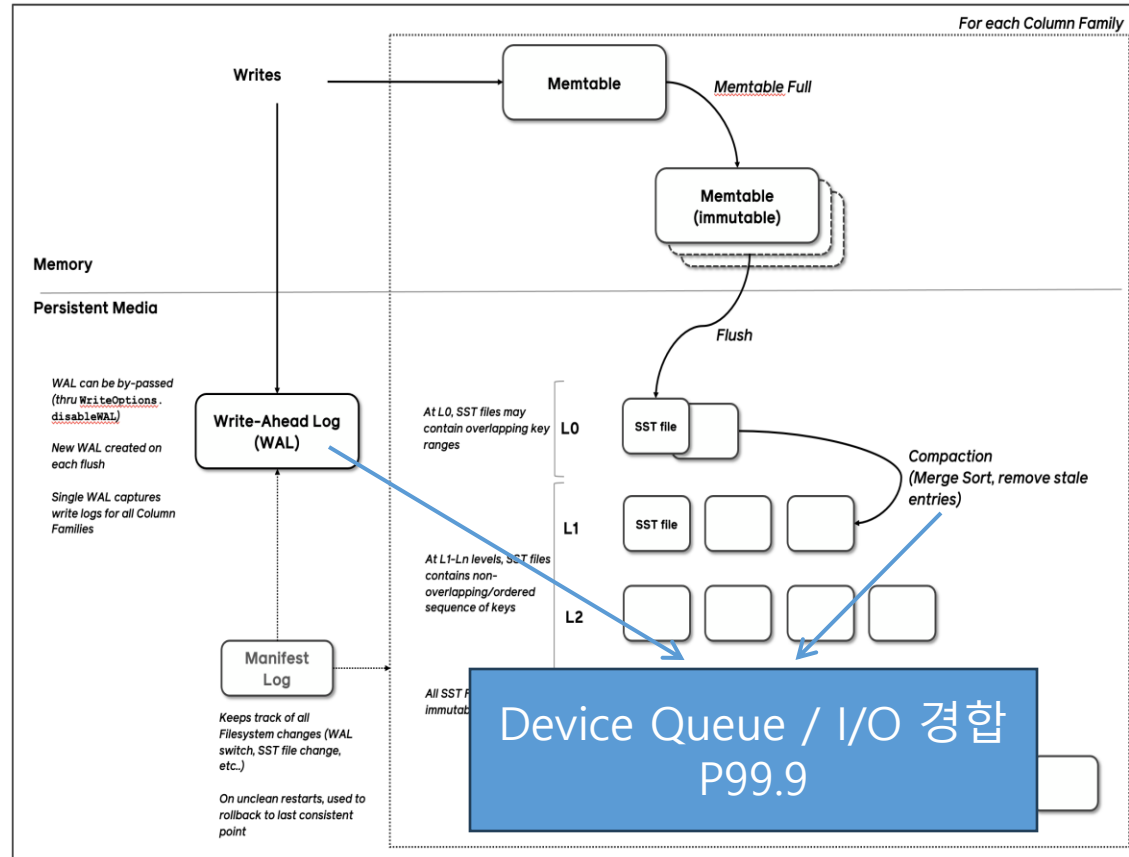
wal_bytes_per_sync

RocksDB는 wal_bytes_per_sync로 일정 바이트마다 fsync 하도록 설정 가능하다. 값을 키워 fsync 횟수를 줄이면 그룹커밋 효과와 함께 fsync 비용을 낮출 수 있으나, 동작 중 크래시 시 일부 데이터 손실 가능성이 커진다.

Rate limiter

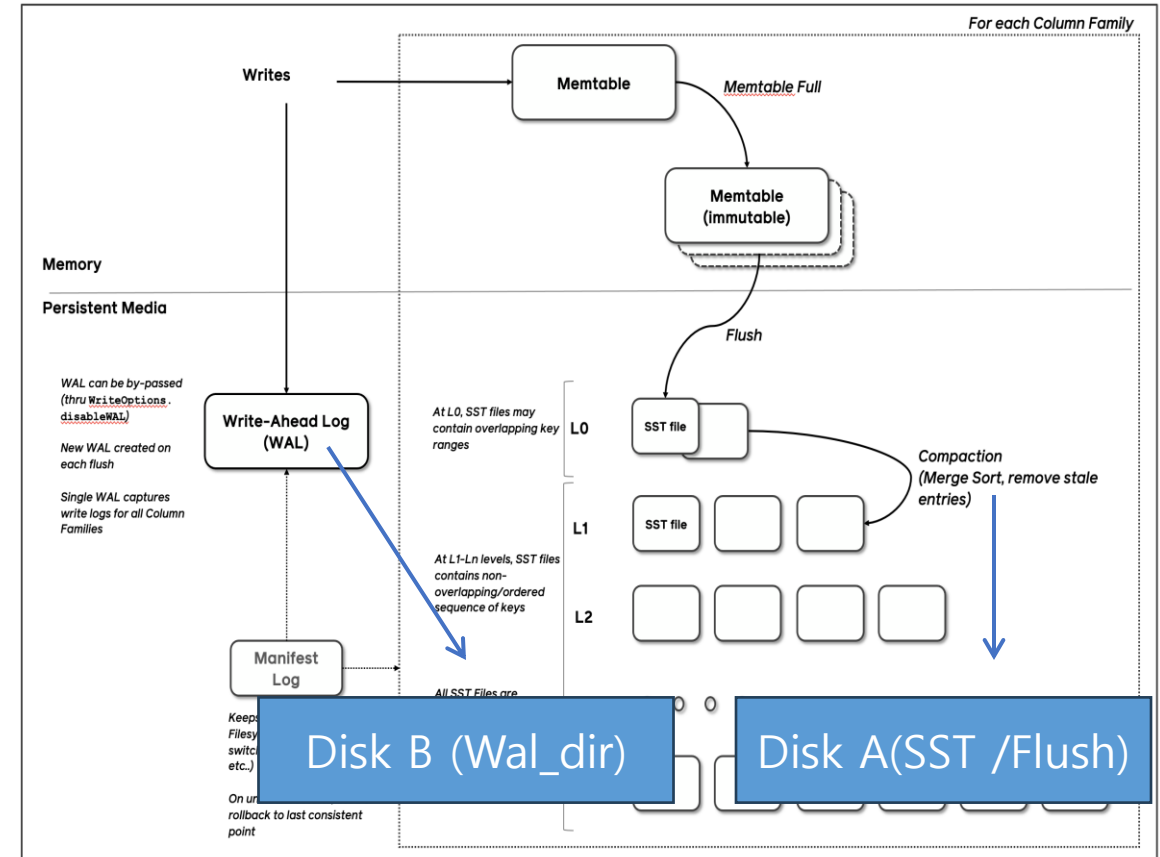
compaction이 디스크를 독점하지 않도록 쓰기 속도를 제한하면 WAL fsync 대기과 갑작스런 latency 스파이크를 완화할 수 있다. 실험으로 적정 bytes/sec 값을 찾아야 한다.

Background – Disk



단일 디스크 사용

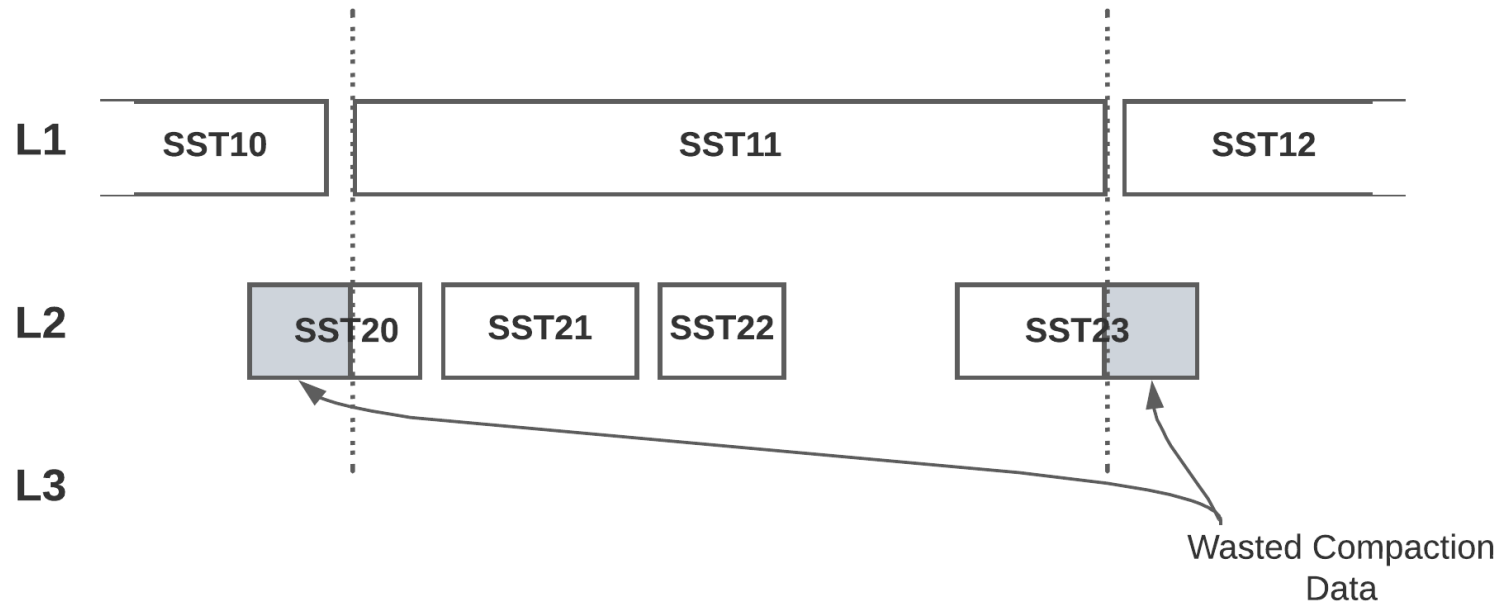
WAL 기록과 SST Flush·Compaction I/O가 같은 디바이스 큐를 공유해 경합이 발생하고 P99.9 지연이 커질 수 있다.



이중 디스크 사용

WAL은 wal_dir로 별도 디스크에 분리하고 SST Flush·Compaction은 데이터 디스크에서 처리해 디바이스 큐 경합을 줄여 P99.9 지연 완화를 노린다.

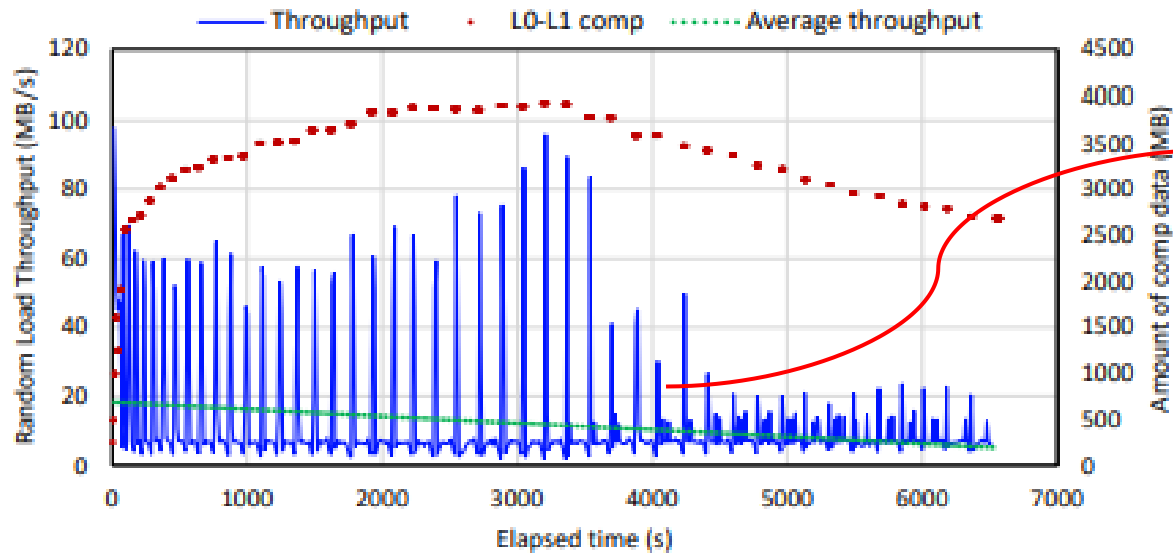
Background - Stall



Stall 발생 (1)

과도한 Compaction I/O로 인해 디스크 대역폭이 소모되면 WAL fsync가 지연되고, 이는 곧 write stall과 tail latency 증가로 이어진다.

Background - Stall



Write stall: Application throughput periodically drop to nearly zero.

- Unpredictable performance.
- Long tail latency.

Stall 발생 (2)

대규모 L0→L1 Compaction은 디스크 I/O를 점유하여 WAL fsync를 지연시키고, 쓰기 처리량 급락(write stall)을 유발한다.

Compaction Rate Limiter로 WAL fsync 경합 완화

Hypothesis

Split I/O (경로)

WAL과 SST 저장 경로를 분리하면 지연시간(p99.9)이 감소할 것이다.

Sync I/O (요청)

wal_bytes_per_sync를 증가시키면, 잦은 fsync 시스템 콜로 인한 I/O 대기열 경합이 감소하여 stall이 개선될 것이다.

Rate (I/O 유속)

Compaction I/O Rate Limiter를 WAL 디스크의 지속 쓰기 대역폭 이하로 제한하면, Compaction이 WAL 쓰기 경로와의 I/O 대역폭을 경쟁하는 현상이 감소하여 fsync 지연이 감소하고 결과적으로 Write Stall 및 p99.9이 감소할 것이다.

Experiment - configuration

- Linux Kernel : 6.14.0-37
- Linux distro : Ubuntu 24.04.03 LTS x86_64
- CPU : Intel 155H @ 4.50
- Memory : LPDDR5 16GB
- Internal Drive : 1TB NVME SSD Micron 2400 SSD 1TB pci4 qlc

Max seq read 4500MB/s, write 3600MB/s

- External Drive : kioxia exceria plus g2 portable ssd 500GB

Max seq read 1050 MB/s write 1000MB/s connected via USB 3.2 gen2 (upto 20Gbps)

- RocksDB : 10.9.1

Experiment – configuration (shell script)

```
9 # [2. 하드웨어 설정]
10 DURATION=100
11 NUM_RUNS=5
12 THREADS=16
13 VALUE_SIZE=400
14 WRITE_BUFFER_SIZE=134217728
15 NUM_KEYS=30000000
16
17 # [3. 옵션 값 정의]
18 SYNC_OPT="--wal_bytes_per_sync=4194304"
19 RATE_OPT="--rate_limiter_bytes_per_sec=734003200"
20
21 COMMON_OPTS=(
22     --benchmarks="fillrandom,stats"
23     --num=${NUM_KEYS}
24     --key_size=16
25     --value_size=${VALUE_SIZE}
26     --write_buffer_size=${WRITE_BUFFER_SIZE}
27     --threads=${THREADS}
28     --max_background_jobs=8
29     --duration=${DURATION}
30     --statistics
31     --histogram=true
32 )
```

Experiment - variables

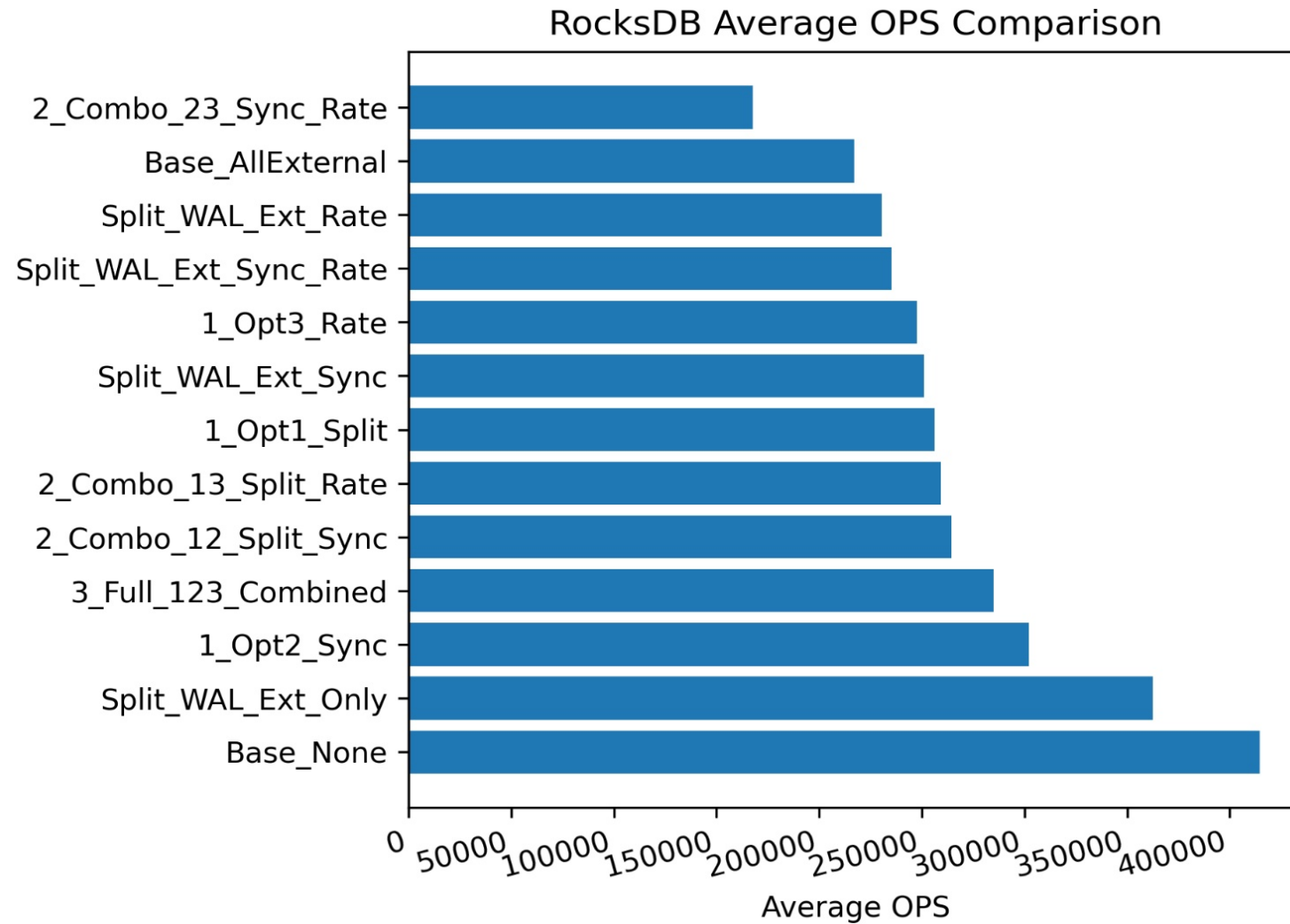
변수 구분	변수 이름	값의 종류	Disk I/O 병목과의 관계
A. 독립 변수 (Input Variables)			
1. SST/WAL 배치	SST_LOCATION / WAL_LOCATION	NVMe, 외장SSD의 5가지 조합	물리적 I/O 채널 분리 및 병목 원인 지정
2. Sync 튜닝	wal_bytes_per_sync	(기본), (4MB 증가)	I/O 요청 횟수(Frequency) 및 오버헤드 감소
3. Rate 튜닝	compaction_rate_limiter	(기본), (700MB/s 전략적 제한)	I/O 대역폭 유속(Rate) 및 Compaction 부하 통제
B. 종속 변수 (Output Variables)			
1. 처리량	Avg OPS	높을수록 좋음	I/O 병목으로 인한 전체적인 처리 능력 저하 정도
2. 지연 시간	P99.9 (us)	낮을수록 좋음	I/O 대기열 경합으로 인해 발생하는 튜는 응답 속도
3. 안정성	Stall %	낮을수록 좋음	I/O 자원 부족으로 인해 쓰기 작업이 강제로 멈춘 시간의 비율 (가장 직접적인 병목 지표)
4. 효율성	WAF	낮을수록 좋음	I/O 비효율성(잘은 재쓰기) 정도
C. 통제 변수			
	THREADS, VALUE_SIZE, DURATION	고정 값	I/O 경합을 유도하고 측정 시간을 통제하는 부하

Experiment – 성능 비교표

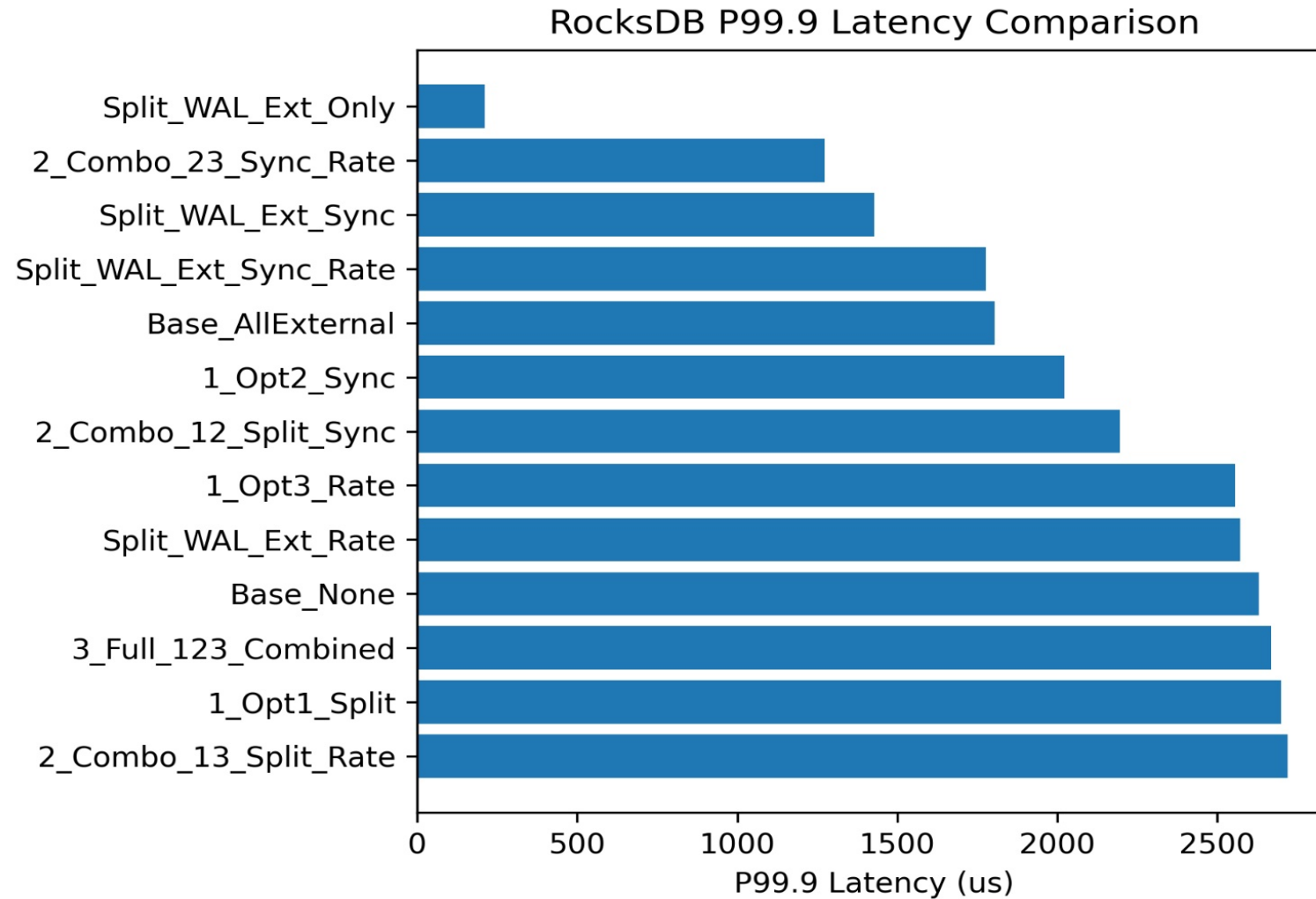
RocksDB 모든 시나리오 통합 성능 비교표

그룹	시나리오 이름	SST 위치	WAL 위치	적용 옵션	Avg OPS	P99.9 (us)	Stall%	WAF
기준점	0_Base_None	NVMe	NVMe	기본	414,694	2630.36	32.60%	2.70
	Ref_AllExternal	외장SSD	외장SSD	기본	217,216	1803.32	58.76%	2.12
아키텍처 A	1_Opt1_Split	외장SSD	NVMe	Split	256,142	2700.19	59.76%	2.40
(로그만 고속)	2_Combo_12_Split_Sync	외장SSD	NVMe	Split+Sync	264,512	2194.75	46.34%	2.46
	2_Combo_13_Split_Rate	외장SSD	NVMe	Split+Rate	259,237	2720.17	59.24%	2.40
	3_Full_123_Combined	외장SSD	NVMe	전종합	284,862	2668.85	49.34%	2.46
아키텍처 B	Split_WAL_Ext_Only	NVMe	외장SSD	Split	362,375	2111.20	36.38%	2.58
(데이터 고속)	Split_WAL_Ext_Sync	NVMe	외장SSD	Split+Sync	250,890	1429.58	31.52%	2.98
	Split_WAL_Ext_Rate	NVMe	외장SSD	Split+Rate	230,510	2570.94	58.50%	2.14
	Split_WAL_Ext_Sync_Rate	NVMe	외장SSD	전종합	235,289	1776.43	37.84%	2.44
S/W 최적화	1_Opt2_Sync	NVMe	NVMe	Sync	302,125	2022.54	17.58%	2.68
(NVMe 기반)	1_Opt3_Rate	NVMe	NVMe	Rate	247,710	2554.92	57.06%	2.20
	2_Combo_23_Sync_Rate	NVMe	NVMe	Sync+Rate	167,718	1273.03	3.00%	2.22

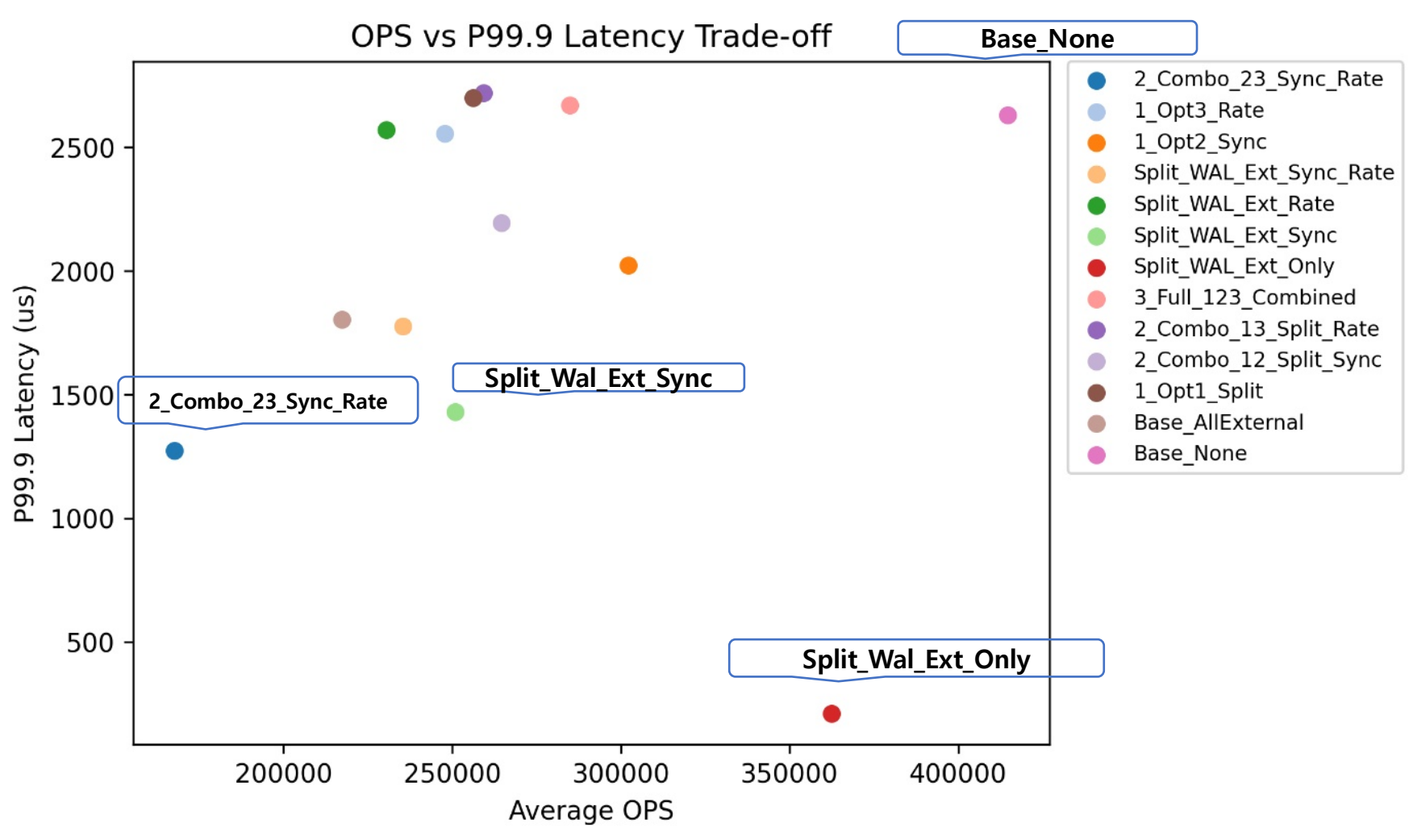
Experiment – Average Ops Comparison



Experiment – P99.9 Latency Comparison



Experiment – P99.9 Latency Trade - off



Result – 주요 옵션 결과

옵션/시나리오	SST 위치	WAL 위치	Stall%	P99.9 (지연시간)	Avg OPS (처리량)	비고
0_Base_None	NVMe	NVMe	32.60%	2630.36us	414,694	NVMe (기준)
1_Opt2_Sync	NVMe	NVMe	17.58%	2022.54us	302,125	낮은 지연 낮은 Stall
Split_WAL_Ext_Only	NVMe	외장SSD	36.38%	2111.20us	362,375	높은 처리량 낮은 지연
1_Opt1_Split	외장SSD	NVMe	59.76%	2700.19	256,142	-
2_Combo_23	NVMe	NVMe	3.00%	1273.03us	167,718	높은 안정성 낮은 처리량

Result

1. 드라이브 분리는 시스템 처리량 개선에 제한적이고 안정성을 높인다.

- 분리를 한다면 SST를 빠른 드라이브에 배치.

2. 안정성(Stall)의 핵심은 Sync 튜닝이다.

3. Rate Limiter는 단독으로 낮은 처리량, 높은 지연을 야기한다.

Result – 아키텍처 별 옵션 선택(옵션 2)

시나리오	SST/WAL 위치	Stall%	P99.9 (us)	결과 분석
0_Base_None	NVMe / NVMe	32.60%	2630.36	fsync 오버헤드로 인해 대기/스톨로 병목 발생
1_Opt2_Sync	NVMe / NVMe	17.58%	2022.54	fsync 빈도 감소 Stall 대폭 완화 tail latency 개선 (최고의 안정화)

주요 통찰(발견)	근거 데이터	의미(해석)	주요 통찰(발견)	근거 데이터
SST가 WAL보다 더 중요	Split_WAL_Ext_Only OPS가 1_Opt1_Split보다 약 41% 높음	Log(WAL)보다 Data(SST)/Compaction I/O를 빠른 디스크(NVMe)에 두는 것이 처리량 유지에 더 결정적	SST가 WAL보다 더 중요	Split_WAL_Ext_Only OPS가 1_Opt1_Split보다 약 41% 높음
Sync 튜닝이 핵심 안정화책	1_Opt2_Sync에서 Stall% 17.58%로 급감	고성능 디스크여도 fsync/동기화 호출을 “덜 자주” 만들어 I/O 오버헤드/대기를 줄이는 것이 안정성에 가장 효과적	Sync 튜닝이 핵심 안정화책	1_Opt2_Sync에서 Stall% 17.58%로 급감

결론 -> 병목은 WAL보다 SST(Compaction)에서 강하게 발생한다.
고성능 드라이브에서는 **I/O 요청 횟수(Sync)를 줄이는 것이** 가장 효과적이다.

Discussion – 목표에 따른 아키텍처 선택

목표	1순위 시나리오	Avg OPS	P99.9	Stall%	선택 이유 및 권장 환경
A. 최대 처리량	0_Base_None	414,694	2630us	32.6%	NVMe의 대역폭 극대화. 높은 Stall 최대한 많은 요청을 빠르게 처리하는 웹 캐시/분석 백엔드에 적합.
B. 최고의 안정성(QoS)	2_Combo_23_Sync_Rate	167,718	1273us	3.00%	금융/거래 시스템 낮은 지연
C. 성능-안정성 밸런스	1_Opt2_Sync	302,125	2022us	17.58%	대부분의 범용 서비스

Discussion – 소프트웨어 튜닝 전략

워크로드 성격	필수 옵션	옵션 2 (Sync)	옵션 3 (Rate)	선택 아키텍처
순수 처리량 (Bulk Write)	없음	비추천 (Stall이 32% 발생해도 총 OPS가 가장 높음)	비추천	0_Base_None (All NVMe)
범용 서비스 (Latency-Sensitive)	Sync 필수	적용 (Stall 17.58%로 급감)	미적용	1_Opt2_Sync (All NVMe) Fast Data + Sync
금융/거래 (High QoS)	Sync & Rate	적용	적용 (Stall 3.0%로 P99.9 방어)	2_Combo_23_Sync_Rate (All NVMe)
자주 멈추는 느린 Disk	Sync 필수	적용	미적용	Ref: AllExternal 상태에서 Stall% 58% → 31%대로 개선 가능

Discussion – 개선사항

USB 프로토콜 드라이브가 아닌, System에 내장 드라이브를 추가 장착 후 실험 필요.

Thank you